

Tutorial 4 - I2S Microphone and STEMMA Speaker on Raspberry Pi

Sound sensing, Raspberry Pi processing, STEMMA speaker playback, and fuzzy audio feedback

This notebook explains how to connect and test an **Adafruit I2S microphone breakout** as the audio input sensor and an **Adafruit Mono Class-D Audio Amplifier with 8 Ω 1 W Speaker** as the physical output device.

The final workflow is:

```
Real sound in the room
      ↓
Adafruit I2S microphone captures sound
      ↓
Raspberry Pi records a WAV file
      ↓
STEMMA speaker plays the recorded audio
      ↓
Python analyzes the waveform
      ↓
Fuzzy logic interprets the sound situation
      ↓
STEMMA speaker gives physical feedback tones
```

1. Learning Objectives

By the end of this activity, you should be able to:

1. Identify the I2S microphone, STEMMA speaker amplifier, Raspberry Pi GPIO header, breadboard, jumper wires, and JST cable.
2. Understand the difference between **digital audio input** through I2S and **audio output** through Raspberry Pi PWM audio.
3. Safely prepare and solder header pins for the microphone breakout.
4. Wire the I2S microphone to the Raspberry Pi using the correct I2S GPIO pins.
5. Wire the STEMMA speaker board to power, ground, and GPIO13 / physical pin 33.
6. Configure the Raspberry Pi audio overlays for the I2S microphone and PWM audio output.
7. Record a WAV file from the microphone using `arecord`.
8. Play the recorded WAV file through the STEMMA speaker using `aplay`.
9. Load and visualize the recorded waveform in Python.

10. Extract interpretable audio features such as loudness, peak level, and impulsiveness.
11. Use fuzzy logic to convert raw audio features into human-friendly STEMMA speaker feedback.

2. Experiment Overview

This experiment has one input path and one output path.

Input path

Sound → I2S microphone → Raspberry Pi GPIO/I2S → WAV recording

Output path

WAV file / Python decision → Raspberry Pi PWM audio / GPIO13 → STEMMA amplifier → speaker sound

The microphone and speaker do not use the same kind of signal. The microphone is a **digital I2S audio input**. The STEMMA speaker board is an **amplified speaker output**. In this setup, Raspberry Pi routes audio playback to GPIO13 using the `audremap` overlay.

You first verify the hardware, then record sound, play it back through the STEMMA speaker, analyze it, and finally generate fuzzy feedback tones.

3. Components Used

3.1 Raspberry Pi Zero 2 W

The Raspberry Pi is the controller. It runs Linux, records audio from the I2S microphone, performs Python analysis, and sends output signals to the STEMMA speaker.

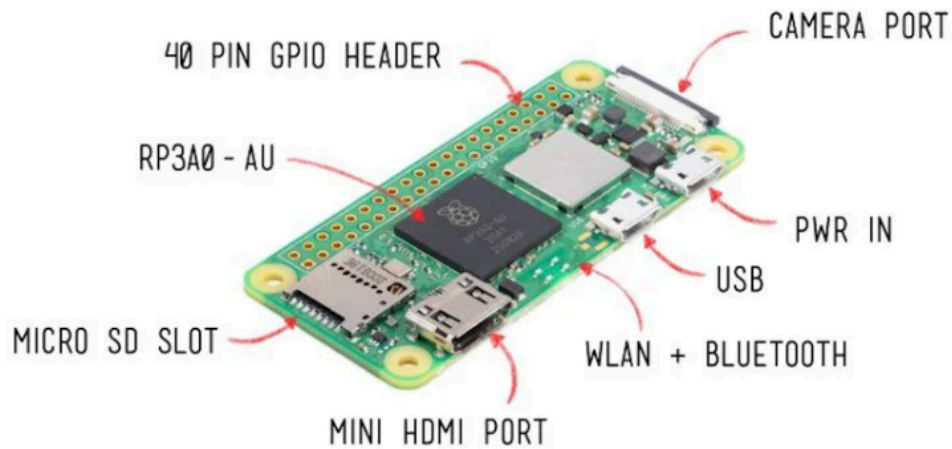


Figure 1: Raspberry Pi

3.2 Breadboard

The breadboard organizes the microphone and jumper connections. The exact rows are flexible, but the pin labels must be followed correctly.

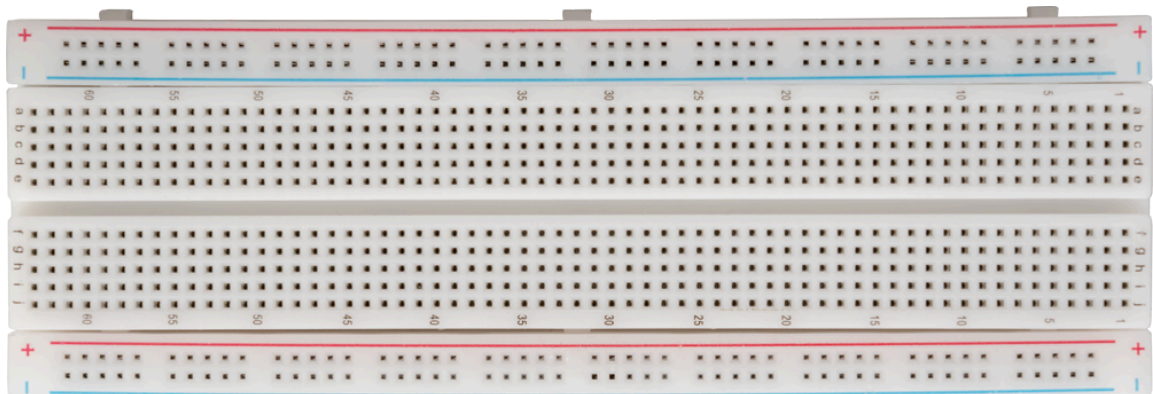
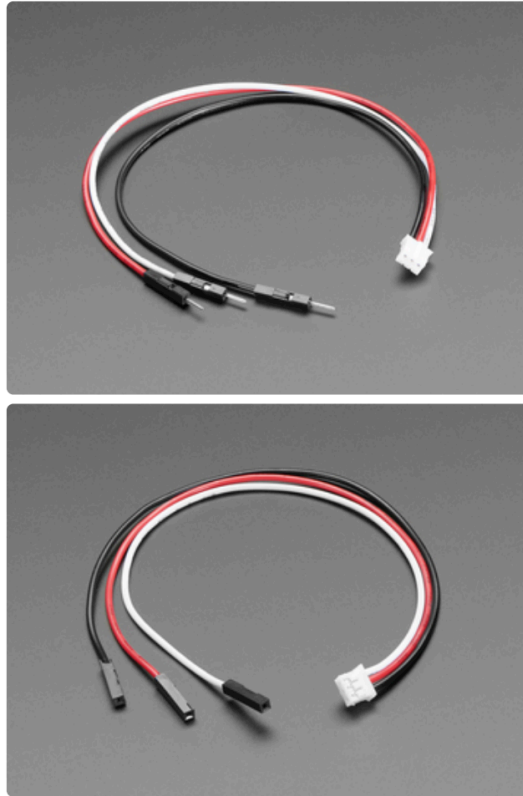


Figure 2: Breadboard

3.3 Jumper Wires and JST Cable

Jumper wires connect the microphone and Raspberry Pi GPIO pins. The STEMMMA speaker uses a 3-pin JST cable labelled through the board as **GND**, **POWER**, and **SIGNAL**.



1. JST PH 3-Pin to Male Header Cable - 200mm
2. JST PH 3-Pin to Female Socket Cable - 200mm

Figure 3: Jumper wires and 3-pin JST cable

3.4 Adafruit I2S Microphone Breakout

The I2S microphone is the input sensor. It captures sound and sends digital audio data to the Raspberry Pi.

Typical pins on the breakout are:

Pin	Meaning
3V	3.3 V power input
GND	Ground
BCLK	I2S bit clock
DOUT	I2S data output from microphone
LRCL / WS	Left/right clock or word select
SEL	Channel selection; connect to GND for this mono setup

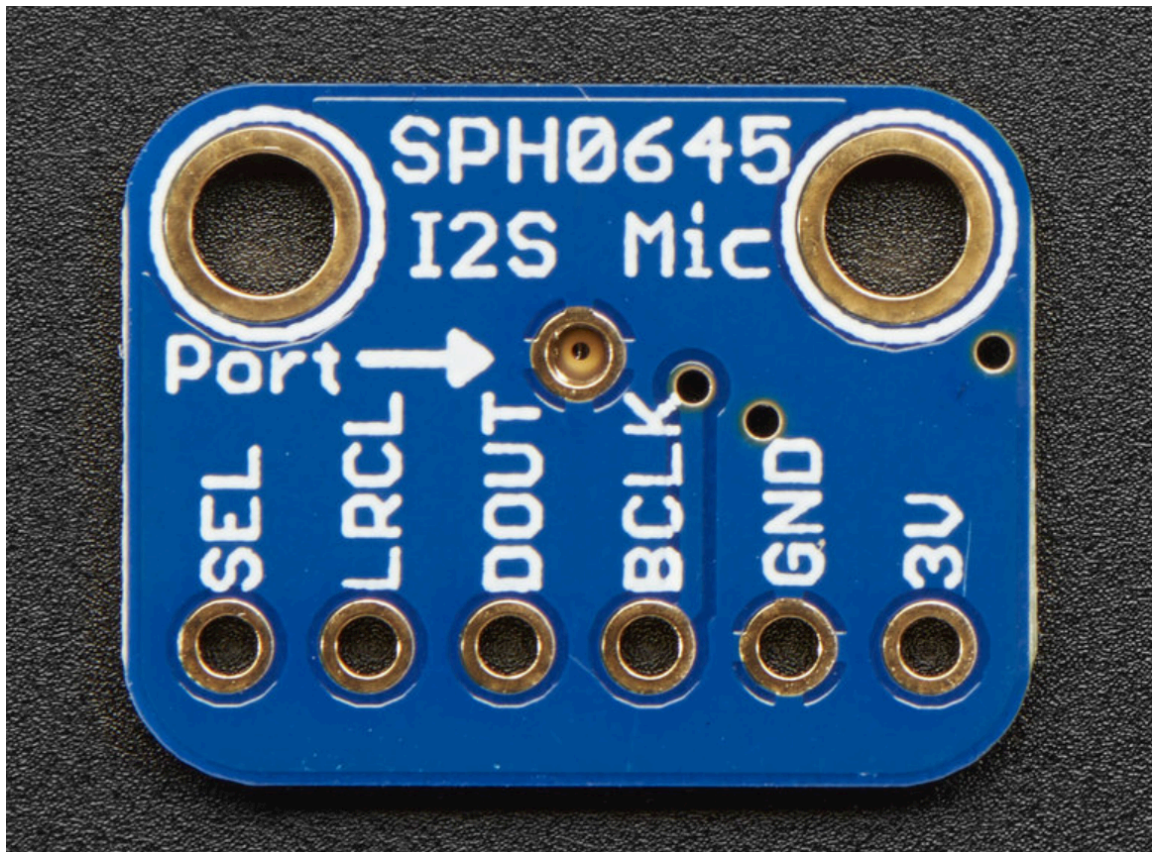


Figure 4: Adafruit I2S microphone breakout with labelled pins

3.5 Adafruit STEMMA Speaker Board

The output component is the **Adafruit Mono Class-D Audio Amplifier with 8 Ω 1 W Speaker**. It is not only a speaker; it already includes an amplifier.

The visible input labels are:

STEMMA Speaker Pin	Function
GND	Common ground with Raspberry Pi
POWER	3 V to 5 V power supply for the amplifier
SIGNAL	Audio/PWM signal input

In this notebook, the STEMMA speaker is used in two ways:

1. to play the recorded WAV file from the microphone, and
2. to produce fuzzy feedback tones from Python.



Figure 5: STEMMA speaker front side showing GND, POWER, and SIGNAL labels

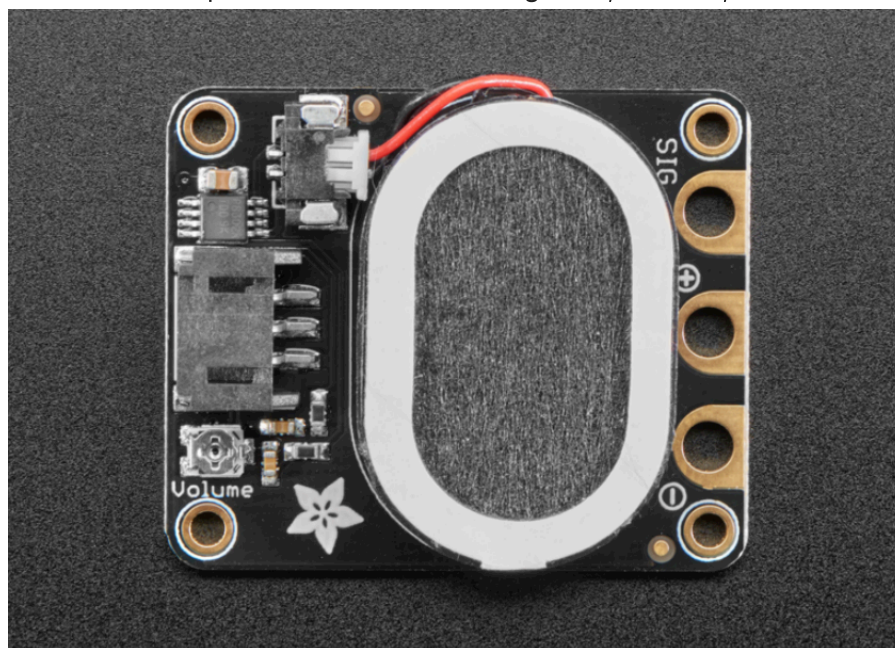


Figure 6: STEMMA speaker back side showing JST connectors and speaker element

4. Safety and Soldering Preparation

4.1 Why soldering is needed

The I2S microphone breakout normally needs header pins soldered to the board before use. Without soldering, electrical contact may be unstable and the microphone may work only when pressed by hand.

Soldering creates:

- stable electrical contact,
- stable mechanical attachment,
- repeatable results,
- fewer false errors during testing.

4.2 Soldering precautions

Follow these precautions:

1. Work on a heat-safe surface.
2. Keep the soldering iron in its stand when not using it.
3. Touch only the insulated handle of the soldering iron.
4. Do not touch the metal tip.
5. Heat the pad and pin briefly, then apply solder.
6. Use only enough solder to form a small shiny cone around the pin.
7. Avoid joining two neighboring pins with solder.
8. Let the board cool before touching the solder joints.
9. Ask for help if solder forms a bridge between pins.

4.3 Recommended soldering orientation

Insert the long side of the header pins into the breadboard. Place the microphone board on the short side of the pins, with the labels visible. Solder each pin from the top side of the board.

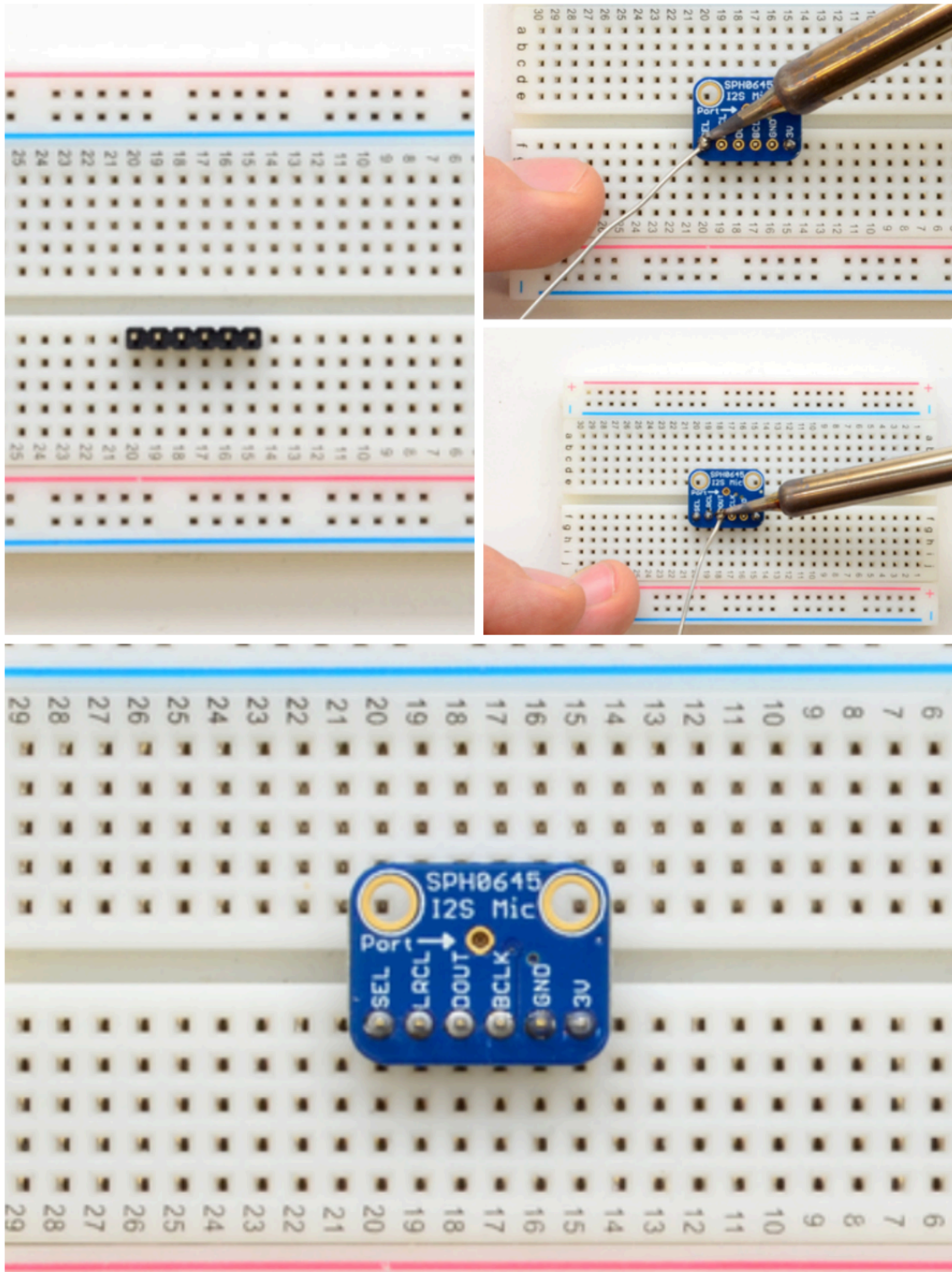


Figure 7: soldering orientation

5. Breadboard Placement and Pin Orientation

The microphone can be placed anywhere on the breadboard. The exact row numbers are not important, but the pin mapping must be clear.

Example placement used in the prototype:

Breadboard Row	Microphone Pin
E50	SEL
E49	LRCL
E48	DOUT
E47	BCLK
E46	GND
E45	3V

6. Raspberry Pi GPIO Wiring

Use the Raspberry Pi Zero 2 W GPIO pinout to identify physical pin numbers.

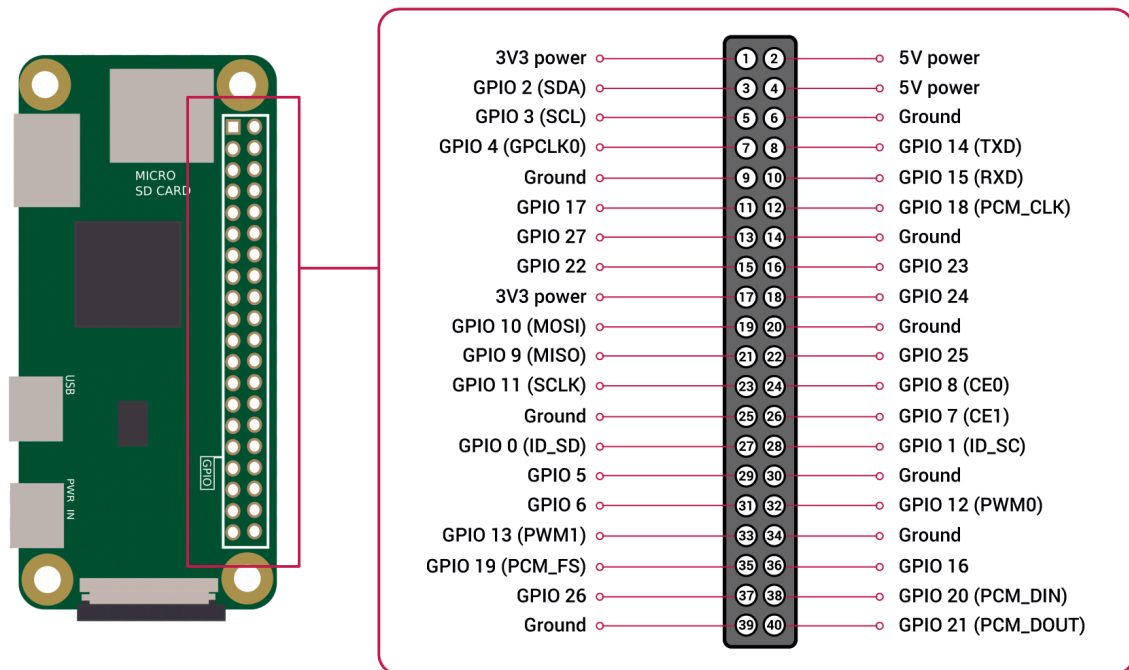


Figure 8: Raspberry Pi Zero 2 W GPIO pinout

6.1 I2S microphone wiring

Connect the microphone as follows:

Microphone Pin	Raspberry Pi Signal	Physical Pin
3V	3.3 V	Pin 1
GND	Ground	Pin 6
BCLK	GPIO18 / PCM_CLK	Pin 12
LRCL	GPIO19 / PCM_FS	Pin 35
DOUT	GPIO20 / PCM_DIN	Pin 38

Microphone Pin	Raspberry Pi Signal	Physical Pin
SEL	Ground	Pin 39

6.2 STEMMA speaker wiring

Connect the STEMMA speaker board as follows:

STEMMA Speaker Pin	Raspberry Pi Signal	Physical Pin
GND	Ground	Pin 34 or Pin 39
POWER	5 V power	Pin 2 or Pin 4
SIGNAL	GPIO13 / PWM1	Pin 33

Important: Do not connect the STEMMA speaker SIGNAL pin to GPIO18 or GPIO19 in this setup. GPIO18 and GPIO19 are already used by the I2S microphone clock signals. GPIO13 is used for STEMMA speaker output.

Power note: The STEMMA speaker board label indicates 3-5 V power. 5 V usually gives a stronger speaker output. The SIGNAL line remains a 3.3 V Raspberry Pi GPIO signal.

7. Raspberry Pi Software Configuration

The I2S microphone and STEMMA speaker playback require Raspberry Pi audio overlays.

The microphone overlay is:

```
dtoverlay=googlevoicehat-soundcard
```

The STEMMA speaker playback overlay is:

```
dtoverlay=audremap,pins_12_13
```

On newer Raspberry Pi OS versions, the configuration file is usually:

```
/boot/firmware/config.txt
```

On older versions, it may be:

```
/boot/config.txt
```

This configuration is a one-time system setup. Editing the file and rebooting should be done in the terminal, not automatically inside the notebook. In this notebook, reboot commands are shown as commented commands so the notebook does not restart your Raspberry Pi by accident.

7.1 Check the operating system version

Run the next cell to identify the OS version.

```
In [ ]: !cat /etc/os-release
```

Expected examples:

```
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
```

or:

```
PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"
```

The prototype worked on Debian 13 / Trixie.

7.2 Check the configuration file location

```
In [ ]: !ls /boot/firmware/config.txt
!ls /boot/config.txt
```

7.3 Check whether the required overlays are already present

```
In [ ]: !grep -E "googlevoicehat|audremap|dtparam=audio" /boot/firmware/config.txt |
```

If one of the required lines is missing, add it manually in the Raspberry Pi terminal:

```
sudo nano /boot/firmware/config.txt
```

A configured setup should contain these lines:

```
dtparam=audio=on
dtoverlay=googlevoicehat-soundcard
dtoverlay=audremap,pins_12_13
```

Save and exit:

```
Ctrl + O
```

```
Enter
```

```
Ctrl + X
```

The reboot command is shown below, but it is intentionally commented out here:

```
# sudo reboot
```

If these lines are already present and the Raspberry Pi was already rebooted after adding them, continue without rebooting.

Software Requirements

This notebook uses Python packages for reading audio data, visualizing waveforms, and controlling GPIO output.

The required Python packages are:

- NumPy
- Matplotlib
- gpiozero

If these packages are not already available in the current Jupyter environment, they can be installed using:

```
pip install numpy matplotlib gpiozero
```

8. Verify the Microphone and Playback Devices

After the overlays are configured and the Raspberry Pi has been rebooted once, check whether the Pi detects the I2S microphone capture device and the PWM audio playback device.

```
In [1]: import subprocess
import re

print("Capture devices:")
capture_output = subprocess.check_output(["arecord", "-l"], text=True)
print(capture_output)

print("\nPlayback devices:")
playback_output = subprocess.check_output(["aplay", "-l"], text=True)
print(playback_output)

# Auto-detect Google VoiceHAT / I2S microphone capture card.
mic_patterns = [
    r"card\s+(\d+):\s+.*google",
    r"card\s+(\d+):\s+.*voice",
    r"card\s+(\d+):\s+.*sndrpigooglevoi",
]

MIC_CARD = None
for pattern in mic_patterns:
    match = re.search(pattern, capture_output, re.IGNORECASE)
    if match:
        MIC_CARD = int(match.group(1))
        break

if MIC_CARD is None:
    MIC_CARD = 2 # fallback from the tested setup
    print("Microphone card was not auto-detected. Using fallback MIC_CARD =")
else:
    print("Selected microphone card:", MIC_CARD)

# After dtoverlay=audremap,pins_12_13, STEMMA playback appears as bcm2835 He
PLAYBACK_CARD = 0
print("Selected STEMMA playback card:", PLAYBACK_CARD)
```

Capture devices:

**** List of CAPTURE Hardware Devices ****

card 2: sndrpigooglevoi [snd_rpi_googlevoicehat_soundcar], device 0: Google voiceHAT SoundCard HiFi voicehat-hifi-0 [Google voiceHAT SoundCard HiFi voicehat-hifi-0]

Subdevices: 1/1

Subdevice #0: subdevice #0

Playback devices:

**** List of PLAYBACK Hardware Devices ****

card 0: Headphones [bcm2835 Headphones], device 0: bcm2835 Headphones [bcm2835 Headphones]

Subdevices: 8/8

Subdevice #0: subdevice #0

Subdevice #1: subdevice #1

Subdevice #2: subdevice #2

Subdevice #3: subdevice #3

Subdevice #4: subdevice #4

Subdevice #5: subdevice #5

Subdevice #6: subdevice #6

Subdevice #7: subdevice #7

card 1: vc4hdmi [vc4-hdmi], device 0: MAI PCM i2s-hifi-0 [MAI PCM i2s-hifi-0]

Subdevices: 1/1

Subdevice #0: subdevice #0

card 2: sndrpigooglevoi [snd_rpi_googlevoicehat_soundcar], device 0: Google voiceHAT SoundCard HiFi voicehat-hifi-0 [Google voiceHAT SoundCard HiFi voicehat-hifi-0]

Subdevices: 1/1

Subdevice #0: subdevice #0

Selected microphone card: 2

Selected STEMMMA playback card: 0

Expected output:

- `arecord -l` should show a Google VoiceHAT / I2S microphone capture device.
- `aplay -l` should show `card 0: Headphones [bcm2835 Headphones]`.

In this setup, `Headphones` is the Raspberry Pi PWM audio output. Because `dtoverlay=audremap,pins_12_13` is enabled, this output is routed to GPIO13 / physical pin 33, where the STEMMMA speaker SIGNAL pin is connected.

Checkpoint

- ☐ Required overlays are present.
- ☐ Raspberry Pi was rebooted once after editing the config file.
- ☐ `arecord -l` shows the Google VoiceHAT / I2S microphone capture device.
- ☐ `aplay -l` shows `bcm2835 Headphones` as a playback device.

9. Record a Test Audio File

This cell records a 5-second WAV file. The recording stops automatically, so you do not need to press `Ctrl + C`.

During recording:

1. Speak clearly near the microphone.
2. Say: "Testing one two three. This is Week 4 microphone test."
3. Clap once or twice.
4. Watch whether the VU meter reacts.

```
In [42]: AUDIO_FILE = "week4_mic_test.wav"
RECORD_SECONDS = 10

record_cmd = (
    f"arecord -D plughw:{MIC_CARD},0 "
    f"-d {RECORD_SECONDS} "
    f"-c1 -r 48000 -f S32_LE -t wav -V mono "
    f"{AUDIO_FILE}"
)

print("Recording command:")
print(record_cmd)
print()
print("Speak clearly near the microphone during recording.")

!{record_cmd}
```

Recording command:

```
arecord -D plughw:2,0 -d 10 -c1 -r 48000 -f S32_LE -t wav -V mono week4_mic_test.wav
```

Speak clearly near the microphone during recording.

Recording WAVE 'week4_mic_test.wav' : Signed 32 bit Little Endian, Rate 48000 Hz, Mono

```
##+ | 03%
```

Confirm that the file was created.

```
In [43]: AUDIO_FILE = "week4_mic_test.wav"

print("Checking recorded file:")
!ls -lh {AUDIO_FILE}
```

Checking recorded file:

```
-rw-r--r-- 1 pi pi 1.9M Jul  1 04:35 week4_mic_test.wav
```

Expected result:

- A file named `week4_mic_test.wav` exists.
- File size is larger than zero.
- The VU meter moved during recording.

Troubleshooting: If the VU meter stays at 0%, check microphone power, ground, BCLK, LRCLK, DOUT, SEL, and soldering.

```
In [44]: import os
from datetime import datetime

AUDIO_FILE = "week4_mic_test.wav"

print("Current notebook folder:")
print(os.getcwd())

print("\nAudio file exists:", os.path.exists(AUDIO_FILE))

if os.path.exists(AUDIO_FILE):
    size = os.path.getsize(AUDIO_FILE)
    modified = datetime.fromtimestamp(os.path.getmtime(AUDIO_FILE))
    print("File size:", round(size / 1024, 1), "KB")
    print("Last modified:", modified)
```

Current notebook folder:

/home/pi/cw_jupyter

Audio file exists: True

File size: 1875.0 KB

Last modified: 2026-07-01 04:35:20.267749

```
In [45]: import wave
import numpy as np

AUDIO_FILE = "week4_mic_test.wav"

with wave.open(AUDIO_FILE, "rb") as wf:
    frames = wf.readframes(wf.getnframes())
    rate = wf.getframerate()
    channels = wf.getnchannels()
    width = wf.getsampwidth()

audio = np.frombuffer(frames, dtype=np.int32)
audio_float = audio.astype(np.float64) / np.iinfo(np.int32).max

rms = np.sqrt(np.mean(audio_float ** 2))
peak = np.max(np.abs(audio_float))

print("Sample rate:", rate)
print("Channels:", channels)
print("Sample width:", width)
print("RMS:", round(float(rms), 6))
print("Peak:", round(float(peak), 6))
```

Sample rate: 48000

Channels: 1

Sample width: 4

RMS: 0.030973

Peak: 0.624908

```
In [48]: import wave
import numpy as np

input_file = "week4_mic_test.wav"
output_file = "week4_mic_test_loud.wav"
gain = 4.0

with wave.open(input_file, "rb") as wf:
    params = wf.getparams()
    frames = wf.readframes(wf.getnframes())

audio = np.frombuffer(frames, dtype=np.int32).astype(np.float64)
audio = audio * gain
audio = np.clip(audio, np.iinfo(np.int32).min, np.iinfo(np.int32).max)
audio = audio.astype(np.int32)

with wave.open(output_file, "wb") as wf:
    wf.setparams(params)
    wf.writeframes(audio.tobytes())

print("Created amplified file:", output_file)
```

Created amplified file: week4_mic_test_loud.wav

```
In [49]: !aplay -D plughw:0,0 week4_mic_test_loud.wav
```

Playing WAVE 'week4_mic_test_loud.wav' : Signed 32 bit Little Endian, Rate 48000 Hz, Mono

```
In [50]: AUDIO_FILE = "week4_mic_test.wav"
!aplay -D plughw:0,0 {AUDIO_FILE}
```

Playing WAVE 'week4_mic_test.wav' : Signed 32 bit Little Endian, Rate 48000 Hz, Mono

```
In [51]: from gpiozero import PWMOutputDevice
from time import sleep

speaker = PWMOutputDevice(13)
speaker.value = 0
sleep(0.2)
speaker.close()

print("GPIO13 quieted.")
```

GPIO13 quieted.

10. STEMMA Speaker Hardware Test: Simple Beep Output

The STEMMA speaker is tested independently first. This confirms that power, ground, signal, and GPIO output are working.

The signal pin is connected to:

GPIO13 / physical pin 33

This is only a hardware beep test. It is not yet recorded voice playback.

```
In [35]: # STEMMMA speaker hardware test using GPIO PWM
# Wiring: STEMMMA SIGNAL -> GPIO13 / physical pin 33

from time import sleep

SPEAKER_GPIO = 13

try:
    from gpiozero import PWMOutputDevice
    print("gpiozero available. Ready to test STEMMMA speaker on GPIO", SPEAKER_GPIO)
except Exception as e:
    print("gpiozero is not available or GPIO cannot be accessed.")
    print("Error:", e)
    print("If needed, install with: sudo apt install -y python3-gpiozero python3-pip")
```

gpiozero available. Ready to test STEMMMA speaker on GPIO 13

```
In [36]: from time import sleep
from gpiozero import PWMOutputDevice

SPEAKER_GPIO = 13

def play_tone(frequency=700, duration=0.35, duty=0.35):
    """Play one PWM tone on the STEMMMA speaker."""
    speaker = PWMOutputDevice(SPEAKER_GPIO, frequency=frequency, initial_value=0)
    try:
        speaker.value = duty
        sleep(duration)
    finally:
        speaker.off()
        speaker.close()

# Simple speaker test pattern
for f in [440, 660, 880]:
    play_tone(frequency=f, duration=0.25, duty=0.35)
    sleep(0.12)

print("STEMMA speaker test complete.")
```

STEMMA speaker test complete.

Expected result:

- Three short tones are heard from the STEMMMA speaker.

If no sound is heard:

1. Check STEMMMA GND is connected to Raspberry Pi GND.
2. Check STEMMMA POWER is connected to 5 V or 3.3 V.
3. Check STEMMMA SIGNAL is connected to GPIO13 / physical pin 33.
4. Check the JST cable orientation.

5. Hold the speaker close for the first test.

Checkpoint

- ☐ STEMMA speaker receives power.
- ☐ STEMMA speaker signal wire is on GPIO13.
- ☐ Speaker test tones are audible.

10.1 Play the Recorded Voice Through the STEMMA Speaker

Now play the recorded WAV file through the STEMMA speaker.

After enabling `dtoverlay=audremap,pins_12_13`, Raspberry Pi exposes the PWM audio output as:

card 0: Headphones [bcm2835 Headphones]

Even though it says `Headphones`, the audio is routed to GPIO13 / physical pin 33, where the STEMMA speaker SIGNAL pin is connected.

```
In [37]: AUDIO_FILE = "week4_mic_test.wav"

play_cmd = f"aplay -D plughw:{PLAYBACK_CARD},0 {AUDIO_FILE}"

print("Playback command:")
print(play_cmd)
print("Listen to the STEMMA speaker for the recorded voice.")

!{play_cmd}
```

Playback command:

`aplay -D plughw:0,0 week4_mic_test.wav`

Listen to the STEMMA speaker for the recorded voice.

Playing WAVE 'week4_mic_test.wav' : Signed 32 bit Little Endian, Rate 48000 Hz, Mono

10.2 Quiet GPIO13 After Playback

If the STEMMA speaker produces a small buzz after playback, run the next cell to release/quiet GPIO13.

```
In [19]: from gpiozero import PWMOutputDevice
from time import sleep

SPEAKER_GPIO = 13

speaker = PWMOutputDevice(SPEAKER_GPIO)
speaker.value = 0
sleep(0.2)
speaker.close()
```

```
print("GPIO13 quieted after playback.")
```

GPIO13 quieted after playback.

11. Load the Recorded Audio in Python

Now inspect the recorded WAV file. This verifies that Python can read the microphone recording.

```
In [20]: from pathlib import Path

wav_path = Path("week4_mic_test.wav")
print("File exists:", wav_path.exists())
print("File size:", wav_path.stat().st_size if wav_path.exists() else "not f
```

File exists: True
File size: 960044 bytes

12. Read and Visualize the Waveform

The waveform shows how the microphone signal changes over time. Peaks usually correspond to speech, claps, taps, or other louder events.

```
In [1]: import wave
import numpy as np
import matplotlib.pyplot as plt

with wave.open(str(wav_path), "rb") as wf:
    n_channels = wf.getnchannels()
    sample_width = wf.getsampwidth()
    sample_rate = wf.getframerate()
    n_frames = wf.getnframes()
    raw_data = wf.readframes(n_frames)

print("Channels:", n_channels)
print("Sample width:", sample_width, "bytes")
print("Sample rate:", sample_rate, "Hz")
print("Frames:", n_frames)

# The arecord command stores signed 32-bit little-endian audio.
audio = np.frombuffer(raw_data, dtype=np.int32)

if n_channels > 1:
    audio = audio.reshape(-1, n_channels)[: , 0]

audio_float = audio.astype(np.float64)
max_abs = np.max(np.abs(audio_float)) if len(audio_float) else 0
if max_abs > 0:
    audio_norm = audio_float / max_abs
else:
    audio_norm = audio_float
```

```
time_axis = np.arange(len(audio_norm)) / sample_rate

plt.figure(figsize=(10, 4))
plt.plot(time_axis, audio_norm)
plt.xlabel("Time (seconds)")
plt.ylabel("Normalized amplitude")
plt.title("Recorded Microphone Waveform")
plt.grid(True)
plt.show()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[1], line 5
      2 import numpy as np
      3 import matplotlib.pyplot as plt
----> 5 with wave.open(str(wav_path), "rb") as wf:
      6     n_channels = wf.getnchannels()
      7     sample_width = wf.getsampwidth()

NameError: name 'wav_path' is not defined
```

13. Extract Audio Features

Instead of using only one raw value, calculate several features from the recording.

Feature	What it describes	Why it matters
RMS level	average signal energy	general loudness
Peak level	strongest moment	sudden clap or sharp event
Activity ratio	how much of the recording is active	continuous noise vs short event
Impulsiveness	peak compared with average energy	detects claps/taps vs normal speech

These features give a more meaningful basis for fuzzy feedback than a single threshold.

```
In [22]: # Basic audio features
abs_audio = np.abs(audio_norm)

peak = float(np.max(abs_audio)) if len(abs_audio) else 0.0
rms = float(np.sqrt(np.mean(audio_norm**2))) if len(audio_norm) else 0.0
activity_ratio = float(np.mean(abs_audio > 0.08)) if len(abs_audio) else 0.0
impulsiveness = float(peak / (rms + 1e-6)) if rms > 0 else 0.0

# Convert features to activity-friendly 0-100 scores.
# These are relative scores, not calibrated decibels.
loudness_score = min(100, rms * 350)
activity_score = min(100, activity_ratio * 250)
impulse_score = min(100, max(0, (impulsiveness - 2.0) * 18))

print(f"Peak level:      {peak:.3f}")
print(f"RMS level:       {rms:.5f}")
print(f"Activity ratio:    {activity_ratio:.3f}")
print(f"Impulsiveness:    {impulsiveness:.2f}")
```

```
print("----")
print(f"Loudness score:    {loudness_score:.1f} / 100")
print(f"Activity score:    {activity_score:.1f} / 100")
print(f"Impulse score:     {impulse_score:.1f} / 100")
```

```
Peak level:      1.000
RMS level:       0.02365
Activity ratio:   0.001
Impulsiveness:   42.28
----
Loudness score:  8.3 / 100
Activity score:  0.2 / 100
Impulse score:   100.0 / 100
```

14. Fuzzy Audio Feedback

This fuzzy experiment does not simply say "low = one beep, high = two beeps." Instead, it combines three ideas:

1. **Loudness** - how strong the sound is overall.
2. **Activity** - whether the sound is continuous or mostly silent.
3. **Impulsiveness** - whether there were sudden sharp events such as claps or taps.

This allows the system to distinguish different acoustic situations:

Situation	Example	Feedback idea
Calm	quiet room	soft confirmation
Active but acceptable	speech/conversation	short neutral tone
Attention needed	loud or sudden sound	stronger warning tone
Strong disturbance	loud and impulsive	repeated alert pattern

```
In [23]: def triangular(x, a, b, c):
          """Triangular membership function."""
          if x <= a or x >= c:
              return 0.0
          if x == b:
              return 1.0
          if x < b:
              return (x - a) / (b - a)
          return (c - x) / (c - b)

          def trapezoid(x, a, b, c, d):
              """Trapezoidal membership function."""
              if x <= a or x >= d:
                  return 0.0
              if b <= x <= c:
                  return 1.0
              if a < x < b:
                  return (x - a) / (b - a)
```

```

    return (d - x) / (d - c)

# Membership degrees for loudness
L = loudness_score
A = activity_score
I = impulse_score

loudness_membership = {
    "low": trapezoid(L, 0, 0, 12, 35),
    "medium": triangular(L, 25, 50, 75),
    "high": trapezoid(L, 60, 80, 100, 100),
}

activity_membership = {
    "brief": trapezoid(A, 0, 0, 10, 35),
    "moderate": triangular(A, 25, 50, 75),
    "continuous": trapezoid(A, 60, 80, 100, 100),
}

impulse_membership = {
    "smooth": trapezoid(I, 0, 0, 15, 40),
    "sharp": triangular(I, 30, 60, 90),
    "very_sharp": trapezoid(I, 75, 90, 100, 100),
}

print("Loudness memberships:", loudness_membership)
print("Activity memberships:", activity_membership)
print("Impulse memberships:", impulse_membership)

```

```

Loudness memberships: {'low': 1.0, 'medium': 0.0, 'high': 0.0}
Activity memberships: {'brief': 1.0, 'moderate': 0.0, 'continuous': 0.0}
Impulse memberships: {'smooth': 0.0, 'sharp': 0.0, 'very_sharp': 0.0}

```

```

In [24]: # Fuzzy rule base
# Each rule returns a feedback class with a strength.

rules = {
    "calm": max(
        min(loudness_membership["low"], activity_membership["brief"]),
        min(loudness_membership["low"], impulse_membership["smooth"]),
    ),
    "normal_activity": max(
        min(loudness_membership["medium"], activity_membership["moderate"]),
        min(loudness_membership["medium"], impulse_membership["smooth"]),
    ),
    "attention": max(
        loudness_membership["high"],
        min(impulse_membership["sharp"], loudness_membership["medium"]),
        min(activity_membership["continuous"], loudness_membership["medium"]),
    ),
    "disturbance": max(
        min(loudness_membership["high"], impulse_membership["sharp"]),
        min(loudness_membership["high"], activity_membership["continuous"]),
        impulse_membership["very_sharp"],
    ),
}

```

```

final_label = max(rules, key=rules.get)
final_strength = rules[final_label]

print("Fuzzy rule strengths:")
for k, v in rules.items():
    print(f"{k:16s}: {v:.3f}")

print("\nFinal fuzzy interpretation:", final_label)
print("Confidence/strength:", round(final_strength, 3))

```

```

Fuzzy rule strengths:
calm           : 1.000
normal_activity : 0.000
attention      : 0.000
disturbance    : 0.000

```

```

Final fuzzy interpretation: calm
Confidence/strength: 1.0

```

15. STEMMA Speaker Feedback from the Fuzzy Result

The final fuzzy result is converted into a physical output pattern.

Fuzzy result	Speaker feedback
calm	one soft low tone
normal_activity	one medium confirmation tone
attention	two higher warning tones
disturbance	three strong alert tones

This creates a complete closed-loop experiment:

sensing → interpretation → physical response

```

In [26]: from time import sleep
         from gpiozero import PWMOutputDevice

         SPEAKER_GPIO = 13

         feedback_patterns = {
             "calm": {"freq": 330, "duration": 0.20, "pulses": 1, "duty": 1},
             "normal_activity": {"freq": 550, "duration": 0.25, "pulses": 1, "duty": 1},
             "attention": {"freq": 850, "duration": 0.22, "pulses": 2, "duty": 1},
             "disturbance": {"freq": 1200, "duration": 0.20, "pulses": 3, "duty": 1}
         }

         def play_feedback(label):
             pattern = feedback_patterns[label]
             for _ in range(pattern["pulses"]):

```

```

speaker = PWMOutputDevice(SPEAKER_GPIO, frequency=pattern["freq"], i
try:
    speaker.value = pattern["duty"]
    sleep(pattern["duration"])
finally:
    speaker.off()
    speaker.close()
    sleep(0.12)

print("Playing STEMMA feedback for:", final_label)
play_feedback(final_label)
print("Feedback complete.")

```

Playing STEMMA feedback for: calm
Feedback complete.

16. Troubleshooting Guide

Problem 1: `arecord -l` does not show the microphone

Check:

- Was `dtoverlay=googlevoicehat-soundcard` added?
- Was the Raspberry Pi rebooted after editing the config file?
- Is the microphone wired correctly?
- Is the microphone powered from 3.3 V, not 5 V?
- Are BCLK, LRCL, and DOUT connected to the correct pins?

Problem 2: VU meter stays at 0%

Check:

- Microphone soldering joints.
- Microphone orientation and pin labels.
- SEL connected to GND.
- BCLK, DOUT, and LRCL connections.
- The microphone opening is not blocked.

Problem 3: Recorded voice does not play through the STEMMA speaker

Check:

- `dtoverlay=audremap,pins_12_13` is present in `/boot/firmware/config.txt`.
- Raspberry Pi was rebooted after adding the overlay.
- `aplay -l` shows `card 0: Headphones [bcm2835 Headphones]`.
- STEMMA SIGNAL is connected to GPIO13 / physical pin 33.

- Playback command uses `aplay -D plughw:0,0 week4_mic_test.wav`.

Problem 4: STEMMA speaker makes no sound during tone tests

Check:

- GND is connected to Raspberry Pi GND.
- POWER is connected to 5 V or 3.3 V.
- SIGNAL is connected to GPIO13 / physical pin 33.
- The JST cable order is correct.
- `gpiozero` is installed.
- The notebook is running on the Raspberry Pi, not on a laptop.

Problem 5: Speaker output is too quiet

Try:

- Use 5 V on the POWER pin.
- Keep the speaker close for the first test.
- Speak closer to the microphone while recording.
- Increase the source volume with a software audio tool later if needed.

Problem 6: Buzzing after playback

Run the quiet GPIO13 cell after playback. If the buzz continues, stop any audio process with:

```
sudo killall aplay
```

or reboot manually from the terminal if needed:

```
# sudo reboot
```

Problem 7: Notebook cell keeps running

The recording cell uses `-d 5`, so it should stop automatically after 5 seconds. If a hardware cell does not stop, use:

```
Kernel → Interrupt Kernel
```

17. Completion Checklist

- ☐ Raspberry Pi SSH/Jupyter connection works.
- ☐ I2S microphone is soldered and mounted.
- ☐ I2S microphone wiring is complete.
- ☐ STEMMA speaker wiring is complete.
- ☐ `dtoverlay=googlevoicehat-soundcard` is enabled.

- ☐ `dtoverlay=audremap,pins_12_13` is enabled.
- ☐ `arecord -l` detects the microphone.
- ☐ `aplay -l` shows `bcm2835 Headphones` playback.
- ☐ A 5-second WAV file is recorded.
- ☐ The recorded WAV file plays through the STEMMA speaker.
- ☐ Python reads the WAV file.
- ☐ Waveform is plotted.
- ☐ STEMMA speaker test tones are audible.
- ☐ Fuzzy interpretation is generated.
- ☐ STEMMA speaker feedback is triggered from the fuzzy result.

18. Summary

This notebook demonstrates a complete Week 4 audio sensing and output workflow using Raspberry Pi.

The I2S microphone captures sound from the environment and saves it as a WAV file. The recorded WAV file is played back through the STEMMA speaker using Raspberry Pi PWM audio routed to GPIO13. Python then reads the waveform and extracts features such as loudness, activity, and impulsiveness. Instead of using a simple fixed threshold, fuzzy logic combines these features into a human-readable interpretation. Finally, the STEMMA Class-D speaker produces physical feedback tones.

The final achieved workflow is:

I2S microphone → Raspberry Pi recording → STEMMA playback → Python analysis → fuzzy interpretation → STEMMA speaker feedback

This connects hardware sensing, signal playback, signal processing, human-centric interpretation, and physical response in one integrated experiment.